

Phase 5 Structured Notes: Deletion in Red-Black Trees and 2-3-4 Trees

0. Big Idea of Phase 5

Phase 5 teaches **deletion**.

Deletion is the hardest part of Red-Black Trees because removing a node can break the tree's balance rules.

The most important idea is:

Deleting a red node is usually easy. Deleting a black node can create a double-black problem.

In a Red-Black Tree, black nodes are important because every path from the root to a null child must have the same number of black nodes. If we remove a black node from only one path, that path becomes short by one black node.

That is why deletion is harder than insertion.

Insertion usually creates this problem:

A red node has a red parent.

Deletion can create this problem:

A path lost a black node.

This missing black node is handled using the idea called **double black**.

Simple memory:

Red deletion = easy.
Black deletion = may create double black.
Double black is fixed using the sibling.

1. Current Progress Before Phase 5

Before Phase 5, you already learned four major ideas.

1.1 Phase 1: 2-3 Trees

A 2-3 tree is a balanced multi-way search tree.

A 2-3 tree node can store:

- 1 key, or
- 2 keys

The important deletion rule was:

```
Never leave a node empty.  
If a node becomes empty:  
1. Borrow first.  
2. Merge second.
```

1.2 Phase 2: 2-3-4 Trees

A 2-3-4 tree is a wider version of a 2-3 tree.

A 2-3-4 tree node can store:

- 1 key
- 2 keys
- 3 keys

The important deletion rule stayed the same:

```
If deletion creates an empty node:  
borrow if possible.  
If borrowing is not possible, merge.
```

1.3 Phase 3: Red-Black Trees

A Red-Black Tree is a Binary Search Tree with color rules.

Each node stores:

- data
- color: red or black
- left child pointer

- right child pointer
- parent pointer

The main Red-Black rules were:

1. Every node is red or black.
2. The root is black.
3. Null children count as black.
4. Every path from root to null has the same black count.
5. A red node cannot have a red parent or red child.
6. New inserted nodes start red.

1.4 Phase 4: Red-Black Trees and 2-3-4 Trees Are the Same Idea

A Red-Black Tree is a 2-3-4 tree drawn in binary-tree form.

The mapping is:

Black node = starts a real 2-3-4 node
 Red node = extra key inside the same 2-3-4 node as its black parent

This mapping is very important for Phase 5.

In Phase 5:

Red-Black rotation = 2-3-4 borrowing
 Red-Black recoloring = 2-3-4 merging

2. What Phase 5 Covers

Phase 5 covers these concepts:

1. Why deletion is harder than insertion
2. Normal BST deletion review
3. Physical deletion from the bottom
4. Internal deletion using predecessor or successor
5. Why the physically deleted node matters
6. Red node deletion
7. Black node deletion
8. Double black
9. Sibling-based repair
10. Case 1: sibling is red
11. Case 2: sibling is black and has a red child

12. Case 3: sibling is black and has black/null children
 13. Red-Black deletion and 2-3-4 deletion mapping
 14. Recoloring as merging
 15. Rotation as borrowing
 16. Deletion dry runs
 17. Conceptual code snippets
 18. Complexity
 19. Common beginner mistakes
 20. Key vocabulary
 21. Mini practice questions
 22. One-page memory sheet
-

3. Why Deletion Is Harder Than Insertion

Insertion adds a new node.

Deletion removes a node.

Adding a node can usually be repaired by checking whether a red-red conflict happened.

Deleting a node can remove a black node from one path. That is more serious because Red-Black Trees depend on equal black depth.

3.1 Red-Black black-depth rule

In a Red-Black Tree:

Every path from root to null must have the same number of black nodes.

Example:

```
    20B
   /
 10R  30R
```

Path 1:

20B -> 10R -> null

Black count:

```
20B = 1
10R = still 1
null = 2
```

Path 2:

```
20B -> 30R -> null
```

Black count:

```
20B = 1
30R = still 1
null = 2
```

Both paths have black count 2.

The tree is valid.

3.2 What goes wrong during deletion?

If we delete a red node, black count does not change.

If we delete a black node, black count may change.

Example:

Before:

```
    40B
   /
 20B  60B
```

Delete 20B.

Now the left path lost one black node.

That breaks the black-depth rule.

This is why black deletion is harder.

4. First Rule: Delete Like a Normal BST First

Before worrying about colors, Red-Black deletion begins like normal Binary Search Tree deletion.

A Red-Black Tree is still a BST, so it must follow this rule:

```
left subtree < node < right subtree
```

So deletion begins by finding the key using normal BST search.

5. BST Deletion Review

There are three normal BST deletion cases.

5.1 Case 1: The node is a leaf

A leaf has no children.

Example:

```
  40
 /
20  60
```

If we delete `20`, we simply remove it.

5.2 Case 2: The node has one child

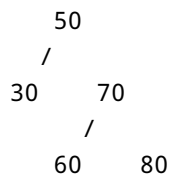
Example:

```
  40
 /
20  60
    70
```

If we delete `60`, we replace it with its child `70`.

5.3 Case 3: The node has two children

Example:



If we delete **70**, we should not simply remove it because it has two children.

Instead, we replace its value with either:

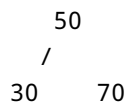
- its predecessor, or
- its successor

Then we physically delete the predecessor or successor from lower in the tree.

6. Internal Deletion

An **internal node** is a node that has children.

Example:



Here, **50** is an internal node because it has children.

In Red-Black Trees, we do not directly remove an internal node with two children.

Instead:

1. Replace the internal node's value with predecessor or successor.
2. Physically delete the predecessor or successor from the bottom.

This keeps the BST order correct.

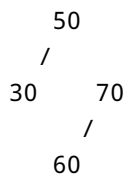
7. Predecessor and Successor

7.1 Predecessor

The predecessor is:

The largest key smaller than the key being deleted.

Example:



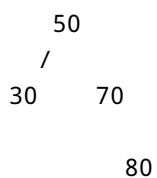
The predecessor of 70 is 60.

7.2 Successor

The successor is:

The smallest key larger than the key being deleted.

Example:



The successor of 70 is 80.

7.3 Why predecessor or successor works

The predecessor and successor are close values.

They keep the BST order correct.

If we replace a deleted key with a nearby value, the tree still remains sorted.

8. Physical Deletion from the Bottom

This is one of the most important Phase 5 ideas.

When deleting a Red-Black node with two children, the key you search for may not be the node that is physically removed.

Example:

```
Delete 100.  
100 is internal.  
Replace 100 with predecessor 90.  
Now physically delete the old 90 from the bottom.
```

The actual node removed is the old `90`, not the original `100` node.

This matters because Red-Black repair depends on this question:

```
What color was the node that was physically removed?
```

Not this question:

```
What color was the key I originally searched for?
```

Beginner memory:

```
Color of searched key is not always the important color.  
Color of physically deleted node is the important color.
```

9. Why the Physically Deleted Node Matters

Suppose you search for `100`, and `100` is black.

You might think:

```
I deleted a black node, so I must fix double black.
```

But maybe `100` was replaced by predecessor `90`, and the old `90` was red.

Then the physical deletion was red.

So no double-black repair is needed.

That is why we always ask:

Which node actually left the tree?
What color was that node?

10. Easy Case: Physically Deleted Node Is Red

If the physically deleted node is red, deletion is easy.

Why?

Because red nodes do not add to black depth.

So removing a red node does not change the black count on any path.

Example:

Before :

50B

70R

Delete 70R .

After :

50B

Black height did not change.

So the deletion is finished.

Simple rule:

Physical red deletion = simple delete.

11. Red Node With a Black Child

A red node usually appears as a leaf in many beginner deletion examples.

But conceptually, if a physically removed red node has a black child, the black child can move up.

The important idea is still:

Removing red does not reduce black height.

So red deletion is still easier than black deletion.

12. Hard Case: Physically Deleted Node Is Black

If the physically deleted node is black, one path may lose a black node.

Example:

Before:

```
      40B
     /
    20B  60B
```

Delete 20B .

The left side lost one black node.

To describe the missing black, we create a temporary idea called **double black**.

13. What Is Double Black?

Double black is not a normal node color.

A Red-Black Tree only has real red and black nodes.

Double black is a temporary repair marker.

It means:

This position is short by one black node.
The tree must fix the black-depth balance here.

Example:

```
      40B
     /
    DB   60B
```

DB means double black.

In simple words:

The left side is missing blackness.

The goal of deletion repair is:

Remove the double black.
Restore equal black depth.
Keep BST order correct.
Keep the root black.

14. Who Helps Fix Double Black?

The sibling helps fix double black.

This is different from insertion.

14.1 In insertion

Insertion usually checks:

```
parent
grandparent
uncle
```

The uncle's color decides whether we recolor or rotate.

14.2 In deletion

Deletion usually checks:

```
double black
parent
sibling
```

The sibling's color and the sibling's children decide what repair to use.

Beginner memory:

```
Insertion problem -> look at uncle.
Deletion problem -> look at sibling.
```

15. Sibling-Based Double-Black Repair

When double black appears, look at the sibling.

There are three main cases.

```
Case 1: Sibling is red.
Case 2: Sibling is black and has at least one red child.
Case 3: Sibling is black and both children are black/null.
```

The repair is either:

```
rotate
```

or

```
recolor
```

In 2-3-4 language:

```
rotate = borrow  
recolor = merge
```

16. Case 1: Sibling Is Red

Suppose the double black has a red sibling.

Example shape:

```
  P  
 /  
DB  SR
```

Where:

```
P  = parent  
DB = double black  
SR = sibling red
```

A red sibling means the sibling side has extra structure.

The repair idea is:

```
Rotate around the parent.  
Adjust colors.  
Then continue fixing if needed.
```

16.1 Why rotation is used

A red sibling is not the final simple case.

Rotation changes the local shape so that we can turn the problem into a more normal case.

Beginner memory:

Sibling red -> rotate first.

16.2 2-3-4 meaning

In 2-3-4 language, a red sibling suggests that the sibling belongs to a larger multi-key node.

That means there may be an extra key available.

So the Red-Black rotation is related to **borrowing**.

17. Case 2: Sibling Is Black and Has a Red Child

Example shape:

```
      P
     /
    DB  SB
      R
```

Where:

```
P  = parent
DB = double black
SB = sibling black
R  = red child of sibling
```

The sibling itself is black, but it has a red child.

That red child means:

The sibling side has an extra key available.

The repair idea is:

Rotate and recolor.

In 2-3-4 language:

Borrow from the sibling through the parent.

Beginner memory:

Sibling black + red child -> rotate -> borrow.

18. Zig-Zig and Zig-Zag During Deletion

Sometimes the sibling's red child is on the outside.

Sometimes it is on the inside.

This can create either a straight-line shape or a zig-zag shape.

The exact rotation depends on the shape.

Simple idea:

If the sibling can lend, rotations rearrange the tree so the borrowed key moves into the correct place.

You do not need to memorize every pointer movement first.

First understand the meaning:

A red child near the sibling means there is something to borrow.
Rotation performs that borrowing in binary-tree form.

19. Case 3: Sibling Is Black and Both Children Are Black/Null

Example shape:

```
  P
 /
DB SB
```



```
 /  
null null
```

The sibling is black.

The sibling has no red child.

That means the sibling cannot lend.

The repair idea is:

```
Recolor the sibling red.  
Let the parent absorb or pass upward the black problem.
```

In many beginner explanations:

```
sibling black -> sibling red  
parent absorbs the extra blackness
```

If the parent cannot absorb it fully, the double-black problem may move upward.

19.1 2-3-4 meaning

In 2-3-4 language, this is merging.

The sibling cannot lend because it has no extra key.

So the empty node, the parent key, and the sibling combine.

Beginner memory:

```
Sibling cannot lend -> recolor -> merge.
```

20. Red-Black Deletion Compared with 2-3-4 Deletion

This is the most important Phase 5 table.

Red-Black deletion idea	2-3-4 deletion idea	Simple meaning
Delete red node	Delete from a node with an extra key	Nothing important is lost
Delete black node	A 2-3-4 node becomes empty	A repair is needed
Double black	Empty-node problem	One side is missing blackness/key support
Sibling has red child	Sibling has extra key	Sibling can lend
Rotate	Borrow	Move help through the parent
Recolor	Merge	Sibling cannot lend, so combine
Problem moves upward	Cascading repair	Parent may now need repair

Simple memory:

Red-Black deletion is 2-3-4 deletion written in binary-tree language.

21. Red Deletion in 2-3-4 Language

In Red-Black form, a red node means:

This key is grouped inside the same 2-3-4 node as its black parent.

So deleting a red node is like deleting one key from a 2-3-4 node that still has another key.

Example 2-3-4 idea:

[30, 50]

Delete 50.

[30]

The node is still valid.

No borrowing or merging is needed.

That matches Red-Black red deletion:

```
Delete red node -> no black-height change.
```

22. Black Deletion in 2-3-4 Language

In Red-Black form, a black node usually represents a real 2-3-4 node.

If deleting a black node empties that 2-3-4 node, we have a problem.

2-3-4 language says:

```
A node became empty.  
Fix it by borrowing or merging.
```

Red-Black language says:

```
A black node was removed.  
Fix double black by rotating or recoloring.
```

These are the same idea in two different forms.

23. Borrowing in 2-3-4 Trees

Borrowing happens when a sibling has an extra key.

Example:

```
Parent key: 20  
Empty node: []  
Sibling: [30, 40]
```

The sibling has extra keys.

So it can lend.

Borrowing idea:

Parent key moves down to fill the empty node.
Sibling key moves up to parent.
Sibling still has at least one key.

This matches Red-Black rotation.

2-3-4 borrow = Red-Black rotate/recolor

24. Merging in 2-3-4 Trees

Merging happens when the sibling cannot lend.

Example:

Parent key: 20
Empty node: []
Sibling: [10]

The sibling has only one key.

It cannot lend.

So we merge:

$[10] + 20 + [] = [10, 20]$

The parent loses a key.

If the parent becomes empty, repair may move upward.

This matches Red-Black recoloring.

2-3-4 merge = Red-Black recolor

25. Why Deletion Repair Can Move Upward

Sometimes a local repair is not enough.

25.1 In 2-3-4 trees

A node becomes empty.

If the sibling cannot lend, we merge.

But merging pulls a key down from the parent.

So the parent loses a key.

If the parent becomes empty, the repair continues upward.

25.2 In Red-Black Trees

A black node is deleted.

Double black appears.

If the sibling cannot help, we recolor.

The black problem may move upward to the parent.

So both trees have the same pattern:

Local repair may cause parent repair.

Beginner memory:

Deletion repair can cascade upward.

26. Full Beginner Red-Black Deletion Algorithm

Here is the beginner version of the algorithm.

```
delete(key):  
  
1. Search for the key like in a normal BST.  
  
2. If the node has two children:  
   Find predecessor or successor.  
   Copy that value into the target node.
```

Now physically delete the predecessor/successor from lower in the tree.

3. Look at the color of the physically deleted node.
4. If the physically deleted node was red:
Delete it.
Stop.
5. If the physically deleted node was black:
A double-black problem may appear.
6. To fix double black:
Look at the sibling.

If sibling is red:
Rotate around the parent.
Adjust colors.

Else if sibling is black and has a red child:
Rotate and recolor.
This is like borrowing in a 2-3-4 tree.

Else if sibling is black and both children are black/null:
Recolor sibling red.
This is like merging in a 2-3-4 tree.
The problem may move upward.
7. Force the root to black.

27. Conceptual BST Delete Code

This is not full production code. It shows the important idea.

```
Node* deleteNode(Node* root, int key) {  
    // Search for the node with the key.  
  
    // If the node has two children:  
    if (node->left != nullptr && node->right != nullptr) {  
        Node* successor = findMin(node->right);  
        node->data = successor->data;  
        node = successor;  
    }  
  
    // Now node is either:
```

```
// 1. a leaf, or
// 2. a node with exactly one child.

// Physically delete this node.
// Red-Black repair depends on this physical node's color.
}
```

Key lesson:

The physical deletion happens at or near the bottom.

28. Conceptual fixDoubleBlack Code

This code shows the main decision logic.

```
void fixDoubleBlack(Node* double_black) {
    Node* sibling = getSibling(double_black);
    Node* parent = double_black->parent;

    // Case 1 and Case 2:
    // Sibling can help, so rotate.
    if (sibling->color == RED || hasRedChild(sibling)) {
        performRotation(parent, sibling);
        adjustColors();
    }

    // Case 3:
    // Sibling cannot help, so recolor.
    else if (sibling->color == BLACK && bothChildrenBlack(sibling)) {
        sibling->color = RED;
        parent->color = BLACK;
    }
}
```

Simple meaning:

Sibling can help -> rotate.
Sibling cannot help -> recolor.

29. Conceptual deleteCorresponding Code

This shows the Red-Black and 2-3-4 connection.

```
void deleteCorresponding(int key) {
    // 2-3-4 Tree Logic
    if (nodeHasExtraKeys()) {
        deleteKey();          // Simple delete
    } else {
        borrowFromSibling();
        if (!canBorrow) {
            mergeNodes();
        }
    }

    // Red-Black Tree Equivalent
    if (node->color == RED) {
        deleteNode();          // Simple delete
    } else {
        if (siblingCanHelp) {
            rotate();           // Mirrors borrowing
        } else {
            recolor();           // Mirrors merging
        }
    }
}
```

Main lesson:

2-3-4 deletion and Red-Black deletion are the same repair idea.
They are just drawn differently.

30. Conceptual fixByRecoloring Code

This case happens when the sibling cannot lend.

```
void fixByRecoloring(Node* double_black) {
    Node* sibling = getSibling(double_black);
    Node* parent = double_black->parent;

    if (sibling->color == BLACK && bothChildrenBlack(sibling)) {
        sibling->color = RED;
    }
}
```



```

        parent->color = BLACK;

        // This mirrors a 2-3-4 merge.
    }
}

```

Simple meaning:

Sibling is black.
 Sibling has no red child.
 Sibling cannot help.
 So recolor.
 This is like merging.

31. Conceptual fixByRotating Code

This case happens when the sibling can lend.

```

void fixByRotating(Node* double_black) {
    Node* sibling = getSibling(double_black);

    if (sibling->color == RED || hasRedChild(sibling)) {
        performRotation();
        adjustColors();

        // This mirrors a 2-3-4 borrow.
    }
}

```

Simple meaning:

Sibling is red or has a red child.
 Sibling can help.
 So rotate and adjust colors.
 This is like borrowing.

32. Dry Run 1: Delete a Red Leaf

Suppose we have:

```

      40B
     /
    20B  60B
      /
     90R

```

Delete 90R .

90 is red.

Red nodes do not add to black depth.

So deletion is simple.

After deletion:

```

      40B
     /
    20B  60B

```

No double black appears.

Rule shown:

Physical red deletion is easy.

33. Dry Run 2: Delete a Black Internal Node Using Predecessor

Suppose we want to delete 100 , and 100 is an internal black node.

The tree uses predecessor 90 .

Steps:

1. Find 100.
2. 100 has children, so do not physically remove it directly.
3. Find predecessor 90.

4. Copy 90 into 100's position.
5. Physically delete the old 90 from the bottom.

If the old physical 90 is red, deletion is easy.

No double black repair is needed.

Rule shown:

The original key may be black, but the physically deleted node may be red.
The physically deleted node's color decides the repair.

34. Dry Run 3: Delete a Black Internal Node Using Successor

Suppose we want to delete 110, and 110 is an internal black node.

The tree uses successor 120.

Steps:

1. Find 110.
2. 110 has children, so do not physically remove it directly.
3. Find successor 120.
4. Copy 120 into 110's position.
5. Physically delete the old 120 from the bottom.

If the old physical 120 is red, deletion is easy.

No double black repair is needed.

Rule shown:

Internal deletion becomes bottom deletion.

35. Dry Run 4: Delete a Black Leaf When Sibling Cannot Lend

Suppose we delete 50B.

Before:

```
    60B
   /
 50B  70B
```

Delete 50B.

Now the left side lost a black node.

```
    60B
   /
  DB   70B
```

Sibling is 70B.

Sibling has no red children.

So sibling cannot lend.

Repair:

```
Recolor sibling 70 from black to red.
Parent 60 absorbs the blackness.
```

After repair idea:

```
    60B
      70R
```

2-3-4 meaning:

Node [50] became empty.
Sibling [70] could not lend.
Parent 60 came down and merged with sibling.
Result: [60, 70]

Rule shown:

Sibling cannot lend -> recolor -> merge.

36. Dry Run 5: Delete a Black Leaf When Sibling Is Red

Suppose we delete 10B, and its sibling is red.

Example idea:

```
      20B
     /
    10B  40R
```

Delete 10B.

```
      20B
     /
    DB   40R
```

Sibling is red.

Repair:

Rotate around the parent.
Adjust colors.
Continue checking if needed.

2-3-4 meaning:

The sibling side has extra structure.
Borrowing is possible.

Rule shown:

Sibling red -> rotate first.

37. Dry Run 6: Delete a Black Leaf When Sibling Is Black with a Red Child

Suppose we delete 10B.

Before:

```
      20B
     /
    10B  40B
      /
     30R
```

Delete 10B.

```
      20B
     /
    DB   40B
      /
     30R
```

Sibling is 40B.

Sibling has red child 30R.

That means sibling can lend.

Repair:

Perform rotation.
Adjust colors.
Restore black height.

2-3-4 meaning:

Empty side needs help.
Sibling [30, 40] has an extra key.
30 can move up while 20 drops down to fill the empty side.

Rule shown:

Sibling black with red child -> rotate -> borrow.

38. Dry Run 7: Delete 80 Black Leaf from the Curriculum Trace

The curriculum includes this idea:

Delete 80 black leaf.

Deleting 80 creates double black.

The sibling is black and has black/null children.

So the sibling cannot lend.

Repair:

Recolor sibling red.
Make/keep parent black.

2-3-4 meaning:

The corresponding 2-3-4 node became empty.
The sibling could not lend.
So the tree merges.

Rule shown:

Black leaf deletion + sibling cannot lend = recolor/merge.

39. Dry Run 8: Delete 90 Black Leaf from the Curriculum Trace

The curriculum also includes this idea:

Delete 90 black leaf.

Deleting 90 creates double black.

The sibling is red.

Repair:

Rotate on the parent.
Adjust colors.
A red-red conflict may appear.
If it appears, use a zig-zag rotation.
Keep the root black.

2-3-4 meaning:

The sibling has extra structure.
Borrowing is possible.

Rule shown:

Sibling can help -> rotate/borrow.

40. Why Rotation Does Not Break BST Order

A rotation changes the shape of the tree.

It does not change the sorted order.

Example:

Before :

10

20

30

After left rotation:

20

/

10

30

The sorted order is still:

$10 < 20 < 30$

So rotation is safe for search order.

In deletion, rotation is used to move help from the sibling side through the parent.

41. Why Recoloring Works

Recoloring changes black counts without changing the tree shape.

When the sibling cannot lend, we often recolor the sibling red.

Why?

Because turning a black sibling red gives away one black count from that side.

This helps balance the missing black on the double-black side.

But sometimes this pushes the problem upward.

That is why recoloring can cause cascading repair.

42. Root Rule After Deletion

At the end of Red-Black deletion repair, the root must be black.

Even if rotations and recoloring happen, always finish with this idea:

```
root->color = BLACK;
```

Simple memory:

```
Always end with a black root.
```

43. Complexity

Even though deletion sounds complicated, it is still efficient.

Why?

Because a Red-Black Tree stays balanced.

The height is logarithmic.

So search, insertion, and deletion stay fast.

Operation	Time Complexity	Simple Meaning
Search	$O(\log n)$	We travel down the height of the tree.
Insert	$O(\log n)$	We search down, then repair upward if needed.
Delete	$O(\log n)$	We search down, then repair upward if needed.
Storage	$O(n)$	We store all nodes.

Rotations and recolorings are local operations.

A single rotation is constant time:

$O(1)$

The logarithmic cost comes from moving down or up the tree height.

Simple memory:

Deletion is complicated, but still $O(\log n)$.

44. Why Phase 5 Is Difficult

Phase 5 is difficult because you must keep three mental models active.

44.1 BST model

Ask:

Where is the key?
Is it a leaf, one-child node, or two-child node?
Do I need predecessor or successor?

44.2 Red-Black color model

Ask:

What color was physically deleted?
Did double black appear?
What color is the sibling?
Does the sibling have a red child?

44.3 2-3-4 model

Ask:

Is this a simple delete?
Can the sibling lend?
Should we borrow?
Should we merge?

Beginner memory:

BST tells you where deletion happens.
Red-Black colors tell you what broke.
2-3-4 meaning tells you why the repair works.

45. Common Beginner Mistakes

Mistake 1: Thinking the original key color always decides repair

Wrong idea:

I searched for a black node, so black repair is always needed.

Correct idea:

The physically deleted node's color decides repair.

If the original key is replaced by a red predecessor or successor, the physical deletion may be red and easy.

Mistake 2: Forgetting to delete from the bottom

Wrong idea:

If a node has two children, remove it directly.

Correct idea:

Replace it with predecessor/successor.
Then physically delete from lower in the tree.

Mistake 3: Confusing uncle and sibling

Insertion uses the uncle.

Deletion uses the sibling.

Memory:

Insertion: parent, grandparent, uncle.
Deletion: double black, parent, sibling.

Mistake 4: Thinking double black is a real color

Double black is not a normal node color.

It is a temporary repair idea.

Real nodes are only:

RED or BLACK

Mistake 5: Rotating when sibling cannot help

If the sibling is black and has no red child, rotation is not the main repair.

The sibling cannot lend.

Use recoloring/merging logic.

Mistake 6: Recoloring when sibling can help

If the sibling is red or has a red child, it can help.

Use rotation/borrowing logic.

Mistake 7: Forgetting null children count as black

Null children are black in Red-Black Trees.

This matters when checking:

Does the sibling have black/null children?

Mistake 8: Forgetting root must stay black

After repairs, always ensure:

```
root is black
```

Mistake 9: Thinking rotation changes sorted order

Rotation does not break BST order.

It only changes the shape.

Mistake 10: Memorizing cases without understanding 2-3-4 meaning

The best shortcut is:

```
Rotate = borrow  
Recolor = merge
```

This makes Red-Black deletion easier to understand.

46. Key Vocabulary

Red-Black Tree

A Binary Search Tree that uses red and black colors to stay balanced.

Binary Search Tree

A tree where smaller values go left and larger values go right.

Red node

A node that does not add to black depth.

In 2-3-4 language, it often represents an extra key grouped with a black parent.

Black node

A node that adds to black depth.

In 2-3-4 language, it usually represents a real separate 2-3-4 node.

Black depth

The number of black nodes on a path from root to null.

Null child

A missing child pointer.

In Red-Black Trees, null children count as black.

Physical deletion

The actual node removed from the tree.

This may be different from the original key searched for.

Internal node

A node that has children.

Predecessor

The largest key smaller than the key being deleted.

Successor

The smallest key larger than the key being deleted.

Double black

A temporary problem marker meaning one path is missing a black node.

Sibling

The other child of the parent.

In deletion, the sibling helps decide the repair.

Rotation

A local tree rearrangement that keeps BST order correct.

In Phase 5, rotation usually means borrowing.

Recoloring

Changing node colors to restore Red-Black rules.

In Phase 5, recoloring usually means merging.

Borrow

A 2-3-4 deletion repair where a sibling gives an extra key through the parent.

Merge

A 2-3-4 deletion repair where an empty node combines with a sibling and a parent key.

Cascading repair

A repair that moves upward because fixing one node creates a problem in its parent.

47. Mini Practice Questions

Question 1

Why is deleting a red node usually easy?

Answer:

Because red nodes do not add to black depth. Removing a red node does not change the number of black nodes on a path.

Question 2

Why is deleting a black node harder?

Answer:

Because removing a black node can make one path lose a black count. This may create double black.

Question 3

What is double black?

Answer:

Double black is a temporary repair marker. It means a position is missing one black node and the tree must restore equal black depth.

Question 4

When deleting an internal node with two children, what should we do first?

Answer:

Replace it with its predecessor or successor. Then physically delete the predecessor or successor from lower in the tree.

Question 5

Which color matters most during Red-Black deletion repair?

Answer:

The color of the physically deleted node.

Question 6

During deletion, should we look at the uncle or the sibling?

Answer:

The sibling.

Insertion usually checks the uncle. Deletion usually checks the sibling.

Question 7

If the sibling is red, what should we do?

Answer:

Rotate around the parent and adjust colors.

Question 8

If the sibling is black and has a red child, what should we do?

Answer:

Rotate and recolor. This is like borrowing in a 2-3-4 tree.

Question 9

If the sibling is black and both children are black/null, what should we do?

Answer:

Recolor the sibling red. This is like merging in a 2-3-4 tree. The problem may move upward.

Question 10

What does Red-Black rotation correspond to in 2-3-4 deletion?

Answer:

Borrowing.

Question 11

What does Red-Black recoloring correspond to in 2-3-4 deletion?

Answer:

Merging.

Question 12

Why can deletion repair move upward?

Answer:

Because merging or recoloring can make the parent lose or absorb blackness. If the parent becomes problematic, repair continues upward.

Question 13

What should be true about the root after deletion repair?

Answer:

The root should be black.

Question 14

What is the time complexity of Red-Black deletion?

Answer:

$O(\log n)$.

The tree height is logarithmic, and repairs move only along the tree height.

48. One-Page Memory Sheet

PHASE 5: DELETION IN RED-BLACK TREES AND 2-3-4 TREES

Big idea:

Deleting red is easy.

Deleting black may create double black.

Step 1:

Delete like a normal BST first.

Step 2:

If the node has two children:

 replace with predecessor or successor.

 physically delete the replacement node from the bottom.

Step 3:

Check the color of the physically deleted node.

If physically deleted node is RED:

 delete it.

 done.

If physically deleted node is BLACK:

 double black may appear.

Double black means:
one path is missing a black node.

To fix double black:
look at the sibling.

Case 1:
Sibling is RED:
rotate around parent.
adjust colors.

Case 2:
Sibling is BLACK and has a RED child:
rotate and recolor.
2-3-4 meaning: borrow.

Case 3:
Sibling is BLACK and both children are black/null:
recolor sibling red.
2-3-4 meaning: merge.
problem may move upward.

Important mapping:
Red deletion = simple delete from extra key.
Double black = empty node problem.
Rotation = borrow.
Recoloring = merge.

Final rule:
root must be black.

Complexity:
Search = $O(\log n)$
Insert = $O(\log n)$
Delete = $O(\log n)$
Storage = $O(n)$

49. Final Beginner Summary

Phase 5 teaches deletion.

Deletion is difficult because removing a black node can break the equal black-depth rule.

The temporary problem is called double black.

To fix double black, look at the sibling.

Use this memory rule:

Sibling can help -> rotate -> borrow.
Sibling cannot help -> recolor -> merge.

Red-Black deletion and 2-3-4 deletion are the same idea in two different forms.

In a 2-3-4 tree, deletion says:

If a node becomes empty, borrow first.
If borrowing is impossible, merge.

In a Red-Black Tree, deletion says:

If black deletion creates double black, check sibling.
If sibling can help, rotate.
If sibling cannot help, recolor.

So the final memory sentence is:

Red-Black deletion is 2-3-4 deletion written in binary-tree language.